

module-18-gui-automation

Chapter 18: Controlling the Keyboard and Mouse with GUI Automation | 36 programs in learning order

Source: Automate the Boring Stuff with Python 1e, Al Sweigart (CC-BY-NC-SA). Syllabus Ch18: pyautogui pause/failsafe, mouse, screenshots, image recognition, keyboard, form filler

How to use this guide (beginner path)

1) Read Concepts once. 2) Follow the Program map for the big picture. 3) Work programs 01..N in order - each builds on the last. [shell] files: type every >>> line into IDLE yourself and compare with the # OUT: comments (errors shown are intentional demos). [script] files: save with the given name and run with F5; predict the output before you run.

Concepts in detail

PyAutoGUI drives the desktop: PAUSE paces calls; FAILSAFE aborts on corner-hit. position()/moveTo()/moveRel() place the cursor; click()/dragTo()/scroll() act; screenshot() captures; pixel()/pixelMatchesColor() inspect; locateOnScreen('btn.png') finds buttons to click. typewrite() sends text char-by-char; press()/keyDown()+keyUp()/hotkey() hit keys. Capstone: read a form's data, tab through fields, handle dropdowns, submit - with pauses so a human can intervene.

Key ideas to carry forward

- pyautogui.PAUSE = 1; pyautogui.FAILSAFE = True - set both first, always.
- locateOnScreen returns (left,top,w,h) or None - check before clicking.
- Form filler skeleton: setup coords -> type fields -> handle selects/radios -> submit + wait.

Classic beginner mistakes

- ! No failsafe/pauses on a fast infinite loop - mouse escapes human control.
- ! locateOnScreen failing on scaled/high-DPI screens - match the screenshot environment.

Study order hint: 01-10 mouse basics + mousenow, 11-18 click/drag/scroll/screenshot, 19-29 recognition + keyboard, 30-36 form filler.

Program map - all 36 at a glance

```
01 01_pauses_fail_safes.py
02 02_controlling_mouse_movement.py
03 03_moving_mouse.py
04 04_moving_mouse_pyautogui.py
05 05_getting_mouse_position.py
06 06_step_1_import.py
07 07_step_2_set.py
08 08_step_3_get.py
09 09_step_3_get_snip.py
10 10_step_3_get_press.py
11 11_clicking_mouse.py
12 12_dragging_mouse.py
13 13_scrolling_mouse.py
14 14_scrolling_mouse_pyperclip.py
15 15_scrolling_mouse_time.py
16 16_getting_screenshot.py
17 17_getting_screenshot_im.py
18 18_analyzing_screenshot.py
19 19_project_extending_mousenow.py
20 20_project_extending_mousenow_press.py
21 21_image_recognition.py
22 22_image_recognition_list.py
23 23_image_recognition_pyautogui.py
```

24 24_sending_string_from.py
25 25_key_names.py
26 26_pressing_releasing_keyboard.py
27 27_hotkey_combinations.py
28 28_hotkey_combinations_pyautogui.py
29 29_hotkey_combinations_pyautogui_2.py
30 30_step_2_set_pyautogui.py
31 31_step_2_set_snip.py
32 32_step_2_set_pyautogui_2.py
33 33_step_3_start.py
34 34_step_3_start_snip.py
35 35_step_4_handle.py
36 36_step_5_submit.py

Topic: Pauses and Fail-Safes

01/36 01_pauses_fail_safes.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import pyautogui
pyautogui.PAUSE = 1
pyautogui.FAILSAFE = True
```

Topic: Controlling Mouse Movement

02/36 02_controlling_mouse_movement.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import pyautogui
pyautogui.size()
# OUT: (1920, 1080)
width, height = pyautogui.size()
```

Topic: Moving the Mouse

03/36 03_moving_mouse.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import pyautogui
for i in range(10):
# OUT:      pyautogui.moveTo(100, 100, duration=0.25)
# OUT:      pyautogui.moveTo(200, 100, duration=0.25)
# OUT:      pyautogui.moveTo(200, 200, duration=0.25)
# OUT:      pyautogui.moveTo(100, 200, duration=0.25)
```

04/36 04_moving_mouse_pyautogui.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import pyautogui
for i in range(10):
# OUT:      pyautogui.moveRel(100, 0, duration=0.25)
# OUT:      pyautogui.moveRel(0, 100, duration=0.25)
# OUT:      pyautogui.moveRel(-100, 0, duration=0.25)
# OUT:      pyautogui.moveRel(0, -100, duration=0.25)
```

Topic: Getting the Mouse Position

05/36 05_getting_mouse_position.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
pyautogui.position()
```

```
# OUT: (311, 622)
```

```
pyautogui.position()
```

```
# OUT: (377, 481)
```

```
pyautogui.position()
```

```
# OUT: (1536, 637)
```

Topic: Step 1: Import the Module

06/36 06_step_1_import.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# mouseNow.py - Displays the mouse cursor's current position.
import pyautogui
print('Press Ctrl-C to quit.')
#TODO: Get and print the mouse coordinates.
```

Topic: Step 2: Set Up the Quit Code and Infinite Loop

07/36 07_step_2_set.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# mouseNow.py - Displays the mouse cursor's current position.
import pyautogui
print('Press Ctrl-C to quit.')
try:
    while True:
        # TODO: Get and print the mouse coordinates.
except KeyboardInterrupt:
    print('\nDone.')
```

Topic: Step 3: Get and Print the Mouse Coordinates

08/36 08_step_3_get.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# mouseNow.py - Displays the mouse cursor's current position.
import pyautogui
print('Press Ctrl-C to quit.')
--snip--
    # Get and print the mouse coordinates.
    x, y = pyautogui.position()
    positionStr = 'X: ' + str(x).rjust(4) + ' Y: ' + str(y).rjust(4)
--snip--
```

09/36 09_step_3_get_snip.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# mouseNow.py - Displays the mouse cursor's current position.
--snip--
    print(positionStr, end='')
    print('\b' * len(positionStr), end='', flush=True)
```

10/36 10_step_3_get_press.py [script]

Script: save and run; predict the output first, then compare.

Press Ctrl-C to quit.

X: 290 Y: 424

Topic: Clicking the Mouse

11/36 11_clicking_mouse.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import pyautogui
pyautogui.click(10, 5)
```

Topic: Dragging the Mouse

12/36 12_dragging_mouse.py [script]

Script: save and run; predict the output first, then compare.

```
import pyautogui, time
time.sleep(5)
pyautogui.click()    # click to put drawing program in focus
distance = 200
while distance > 0:
    pyautogui.dragRel(distance, 0, duration=0.2)    # move right
    distance = distance - 5
    pyautogui.dragRel(0, distance, duration=0.2)    # move down
    pyautogui.dragRel(-distance, 0, duration=0.2)    # move left
    distance = distance - 5
    pyautogui.dragRel(0, -distance, duration=0.2)    # move up
```

Topic: Scrolling the Mouse

13/36 13_scrolling_mouse.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
pyautogui.scroll(200)
```

14/36 14_scrolling_mouse_pyperclip.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
import pyperclip  
numbers = ''  
for i in range(200):  
# OUT:      numbers = numbers + str(i) + '\n'  
  
pyperclip.copy(numbers)
```

15/36 15_scrolling_mouse_time.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import time, pyautogui
time.sleep(5); pyautogui.scroll(100)
```

Topic: Getting a Screenshot

16/36 16_getting_screenshot.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import pyautogui
im = pyautogui.screenshot()
```

17/36 17_getting_screenshot_im.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
im.getpixel((0, 0))
```

```
# OUT: (176, 176, 175)
```

```
im.getpixel((50, 200))
```

```
# OUT: (130, 135, 144)
```

Topic: Analyzing the Screenshot

18/36 18_analyzing_screenshot.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import pyautogui
im = pyautogui.screenshot()
im.getpixel((50, 200))
# OUT:      (130, 135, 144)
pyautogui.pixelMatchesColor(50, 200, (130, 135, 144))
# OUT:      True
pyautogui.pixelMatchesColor(50, 200, (255, 135, 144))
# OUT:      False
```

Topic: Project: Extending the mouseNow Program

19/36 19_project_extending_mousenow.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# mouseNow.py - Displays the mouse cursor's current position.
--snip--
    positionStr = 'X: ' + str(x).rjust(4) + ' Y: ' + str(y).rjust(4)
    pixelColor = pyautogui.screenshot().getpixel((x, y))
    positionStr += ' RGB: (' + str(pixelColor[0]).rjust(3)
    positionStr += ', ' + str(pixelColor[1]).rjust(3)
    positionStr += ', ' + str(pixelColor[2]).rjust(3) + ')'
    print(positionStr, end='')
--snip--
```

20/36 20_project_extending_mousenow_press.py [script]

Script: save and run; predict the output first, then compare.

Press Ctrl-C to quit.

X: 406 Y: 17 RGB: (161, 50, 50)

Topic: Image Recognition

21/36 21_image_recognition.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import pyautogui
pyautogui.locateOnScreen('submit.png')
# OUT: (643, 745, 70, 29)
```

22/36 22_image_recognition_list.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
list(pyautogui.locateAllOnScreen('submit.png'))  
# OUT: [(643, 745, 70, 29), (1007, 801, 70, 29)]
```

23/36 23_image_recognition_pyautogui.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
pyautogui.locateOnScreen('submit.png')  
# OUT: (643, 745, 70, 29)  
pyautogui.center((643, 745, 70, 29))  
# OUT: (678, 759)  
pyautogui.click((678, 759))
```

Topic: Sending a String from the Keyboard

24/36 24_sending_string_from.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
pyautogui.click(100, 100); pyautogui.typewrite('Hello world!')
```

Topic: Key Names

25/36 25_key_names.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
pyautogui.typewrite(['a', 'b', 'left', 'left', 'x', 'y'])
```

Topic: Pressing and Releasing the Keyboard

26/36 26_pressing_releasing_keyboard.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
pyautogui.keyDown('shift'); pyautogui.press('4'); pyautogui.keyUp('shift')
```

Topic: Hotkey Combinations

27/36 27_hotkey_combinations.py [script]

Script: save and run; predict the output first, then compare.

```
pyautogui.keyDown('ctrl')  
pyautogui.keyDown('c')  
pyautogui.keyUp('c')  
pyautogui.keyUp('ctrl')
```

28/36 28_hotkey_combinations_pyautogui.py [script]

Script: save and run; predict the output first, then compare.

```
pyautogui.hotkey('ctrl', 'c')
```

29/36 29_hotkey_combinations_pyautogui_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
import pyautogui, time  
def commentAfterDelay():  
# OUT:      pyautogui.click(100, 100)  
# OUT:      pyautogui.typewrite('In IDLE, Alt-3 comments out a line.')  
# OUT:      time.sleep(2)  
# OUT:      pyautogui.hotkey('alt', '3')  
  
commentAfterDelay()
```

Topic: Step 2: Set Up Coordinates

30/36 30_step_2_set_pyautogui.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# formFiller.py - Automatically fills in the form.

import pyautogui, time

# Set these to the correct coordinates for your computer.
nameField = (648, 319)
submitButton = (651, 817)
submitButtonColor = (75, 141, 249)
submitAnotherLink = (760, 224)

# TODO: Give the user a chance to kill the script.

# TODO: Wait until the form page has loaded.

# TODO: Fill out the Name Field.

# TODO: Fill out the Greatest Fear(s) field.

# TODO: Fill out the Source of Wizard Powers field.

# TODO: Fill out the Robocop field.

# TODO: Fill out the Additional Comments field.

# TODO: Click Submit.

# TODO: Wait until form page has loaded.

# TODO: Click the Submit another response link.
```

31/36 31_step_2_set_snip.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# formFiller.py - Automatically fills in the form.

--snip--

formData = [{'name': 'Alice', 'fear': 'eavesdroppers', 'source': 'wand',
              'robocop': 4, 'comments': 'Tell Bob I said hi.'},
             {'name': 'Bob', 'fear': 'bees', 'source': 'amulet', 'robocop': 4,
              'comments': '\n/a'},
             {'name': 'Carol', 'fear': 'puppets', 'source': 'crystal ball',
              'robocop': 1, 'comments': 'Please take the puppets out of the
              break room.'},
             {'name': 'Alex Murphy', 'fear': 'ED-209', 'source': 'money',
              'robocop': 5, 'comments': 'Protect the innocent. Serve the public
              trust. Uphold the law.'},
             ]

--snip--
```

32/36 32_step_2_set_pyautogui_2.py [script]

Script: save and run; predict the output first, then compare.

```
pyautogui.PAUSE = 0.5
```

Topic: Step 3: Start Typing Data

33/36 33_step_3_start.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

OUT: #! python3

OUT: # formFiller.py - Automatically fills in the form.

OUT: --snip--

OUT: for person in formData:

OUT: # Give the user a chance to kill the script.

OUT: print('>>> 5 SECOND PAUSE TO LET USER PRESS CTRL-C <<<')

OUT: time.sleep(5)

OUT: # Wait until the form page has loaded.

OUT: while not pyautogui.pixelMatchesColor(submitButton[0], submitButton[1],

OUT: submitButtonColor):

OUT: time.sleep(0.5)

OUT: --snip--

34/36 34_step_3_start_snip.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# formFiller.py - Automatically fills in the form.

--snip--

print('Entering %s info...' % (person['name']))
pyautogui.click(nameField[0], nameField[1])

# Fill out the Name field.
pyautogui.typewrite(person['name'] + '\t')

# Fill out the Greatest Fear(s) field.
pyautogui.typewrite(person['fear'] + '\t')

--snip--
```

Topic: Step 4: Handle Select Lists and Radio Buttons

35/36 35_step_4_handle.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# formFiller.py - Automatically fills in the form.

--snip--

# Fill out the Source of Wizard Powers field.
if person['source'] == 'wand':
    pyautogui.typewrite(['down', '\t'])
elif person['source'] == 'amulet':
    pyautogui.typewrite(['down', 'down', '\t'])
elif person['source'] == 'crystal ball':
    pyautogui.typewrite(['down', 'down', 'down', '\t'])
elif person['source'] == 'money':
    pyautogui.typewrite(['down', 'down', 'down', 'down', '\t'])

# Fill out the Robocop field.
if person['robocop'] == 1:
    pyautogui.typewrite([' ', '\t'])
elif person['robocop'] == 2:
    pyautogui.typewrite(['right', '\t'])
elif person['robocop'] == 3:
    pyautogui.typewrite(['right', 'right', '\t'])
elif person['robocop'] == 4:
    pyautogui.typewrite(['right', 'right', 'right', '\t'])
elif person['robocop'] == 5:
    pyautogui.typewrite(['right', 'right', 'right', 'right', '\t'])

--snip--
```

Topic: Step 5: Submit the Form and Wait

36/36 36_step_5_submit.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# formFiller.py - Automatically fills in the form.

--snip--

# Fill out the Additional Comments field.
pyautogui.typewrite(person['comments'] + '\t')

# Click Submit.
pyautogui.press('enter')

# Wait until form page has loaded.
print('Clicked Submit.')
time.sleep(5)

# Click the Submit another response link.
pyautogui.click(submitAnotherLink[0], submitAnotherLink[1])
```